



TECHNISCHE
UNIVERSITÄT
WIEN

MASTER'S THESIS

Development of a Framework for System Simulation

In fulfillment of the requirements for the academic degree

Master of Science

as part of the study program

Computational Science and Engineering

submitted by

Florian Frech, BSc

carried out at the Institute of Lightweight Design and Structural Biomechanics
at the TU Wien

under supervision of

Univ.Prof. Dipl.-Ing. Dr.techn. Dieter Pahr

and with collaboration of

Univ.Lektor Dipl.-Ing. Dr.techn. Harald Sima

March 17, 2026

Abstract

An abstract should concisely summarize your research by first stating the motivation or problem your work addresses, followed by the specific objectives or research questions you aimed to answer.

Next, briefly describe the key methods or approaches you employed, then present the most significant results or findings obtained. Finally, conclude by highlighting the main conclusions and the broader significance or implications of your research. Ensure the abstract is self-contained, clear, concise (typically 150-300 words), and includes relevant keywords.

Acknowledgements

Express your sincere and personal gratitude to everyone who supported you during your Master's thesis, even those not formally involved. Start with your supervisor and, if applicable, your co-supervisor, then mention family and friends for their support and encouragement. Briefly acknowledge specific help, such as proofreading or discussions. Keep it concise and write from the heart.

Typical Phrases:

Supervisor/Co-supervisor:

- "A special thank you goes to my supervisor, Prof. [Name], for their excellent guidance, valuable advice, and constructive discussions throughout this work."
- "I would like to sincerely thank my co-supervisor, Dr. [Name], for their dedicated support and helpful suggestions."

Family and Friends:

- "To my family, thank you from the bottom of my heart for your endless patience, support, and encouragement during this intense period."
- "A big thank you to my friends who always motivated and supported me."
- "I would like to thank [Name] for proofreading and their helpful comments."
- "Thanks to [Name] for the stimulating conversations and mental support."

Further Support:

- "I thank [Person/Institution] for providing [resources/data]."
- "Many thanks to [Person] for their technical assistance."

Concluding Remarks:

- "This thesis would not have been possible without your support. Thank you!"

Acronyms

Abstract

An abstract should concisely summarize your research by first stating the motivation or problem your work addresses, followed by the specific objectives or research questions you aimed to answer.

Next, briefly describe the key methods or approaches you employed, then present the most significant results or findings obtained. Finally, conclude by highlighting the main conclusions and the broader significance or implications of your research. Ensure the abstract is self-contained, clear, concise (typically 150-300 words), and includes relevant keywords.

Kurzfassung

Deine Kurzfassung fasst deine Forschungsarbeit prägnant zusammen, indem sie zuerst die Motivation oder das Problem deiner Arbeit darlegt, gefolgt von den spezifischen Zielen oder Forschungsfragen, die du beantworten wolltest.

Beschreibe anschließend kurz die wichtigsten Methoden oder Ansätze, die du verwendet hast, und präsentiere dann die wichtigsten Ergebnisse oder Erkenntnisse. Abschließend fasst du die wichtigsten Schlussfolgerungen und die breitere Bedeutung oder Implikationen deiner Forschung zusammen. Stelle sicher, dass die Kurzfassung in sich geschlossen, klar, prägnant (in der Regel 100-200 Wörter, prüfe spezifische Richtlinien) ist und relevante Schlüsselwörter enthält.

Contents

1	Introduction	1
1.1	Context and Motivation	1
1.2	State of the Art	2
1.2.1	Equation-based Modeling and FMI-based Components	2
1.2.2	Domain-Specific Simulation Environments	3
1.2.3	Co-Simulation Frameworks and Orchestration	4
1.2.4	Hybrid Co-Simulation and Event Handling	5
1.2.5	Multi-Model and Multi-Fidelity Simulation	6
1.3	Problem Statement and Research Gap	7
1.4	Research Questions	7
1.5	Objectives	8
1.6	Thesis Outline	9
2	Theoretical Background	10
2.1	Continuous-Time System Modeling	10
2.1.1	Dynamical Systems with Inputs and Outputs	10
2.1.2	Ordinary and Differential-Algebraic Equations	11
2.1.3	Direct Feedthrough and Interface Variables	12
2.2	Hybrid System Modeling	13
2.2.1	Hybrid System Representation	13
2.2.2	Events, Zero-Crossings, and State Re-Initialization	14
2.2.3	Time Semantics in Hybrid Simulation	15
2.3	Co-Simulation Principles	16
2.3.1	Functional Mock-up Interface and Functional Mock-up Units	16
2.3.2	Orchestrated Co-Simulation and Communication Steps	16
2.3.3	Dependency Analysis and Execution Order	16
2.3.4	Algebraic Loops in Coupled Systems	16
2.3.5	Hybrid Co-Simulation and Event Handling	16
2.4	Domain-Specific Modelling Foundations	16
2.4.1	Equation-Based Modelling with Modelica	16
2.4.2	Musculoskeletal Modelling with OpenSim	16

2.4.3	Finite Element Analysis with NGSolve	16
3	Requirements and Tool Selection	17
3.1	Requirements	17
3.1.1	Functional User Requirements	17
3.1.2	Non-Functional User Requirements	17
3.1.3	Derived System Requirements	17
3.2	Tool Comparison	17
3.2.1	Equation-Based Modelling	17
3.2.2	Musculoskeletal Modelling	18
3.2.3	Finite Element Analysis Tools	18
3.2.4	System Simulation and Co-Simulation Frameworks	18
4	Framework Architecture	19
4.1	Design Principles and Decisions	19
4.2	Layered Architecture	19
4.3	CoSimComponent Interface	19
4.3.1	Lifecycle	19
4.3.2	Ports	19
4.3.3	State Management	19
4.4	System Class	19
4.4.1	Component Registration	19
4.4.2	Connections	19
4.4.3	Graph Construction	19
4.5	Algorithm Selection	19
5	Feature Implementation and Verification	20
5.1	Port System and Unit-Aware Connections	20
5.2	Component Wrappers	20
5.2.1	FMI 2.0 Co-Simulation Wrapper	20
5.2.2	Musculoskeletal Model Wrapper	20
5.2.3	Finite Element Model Wrapper	20
5.3	Structural Analysis and Execution Ordering	20
5.4	Master Algorithms	20
5.5	Algebraic Loop Resolution	20
5.6	Hybrid Co-Simulation Algorithm	20
5.7	Multi-Model Component and Mode Switching	20
5.8	System Graph Visualization	20
6	Case Study: Controlled Pendulum System	21
6.1	Choice and Justification of the Case Study	21
6.2	System Engineering of the Controlled Pendulum	21
6.2.1	System Architecture and Signal Flow	21

6.2.2	PID Controller	21
6.2.3	BLDC Drive Model	21
6.2.4	Sensors and Quantization	21
6.3	Pendulum Models	21
6.3.1	Modelica Pendulum	21
6.3.2	OpenSim Pendulum	21
6.3.3	FEM Pendulum	21
6.3.4	Multi-Model Pendulum	22
6.4	System Assembly and Configuration	22
6.4.1	Component Wiring and Connection Graph	22
6.4.2	Algebraic Loop Analysis	22
6.4.3	Monolithic Modelica Reference Model	22
7	Discussion and Conclusion	23
7.1	Discussion of Results	23
7.2	Limitations	23
7.3	Conclusions	23
7.4	Outlook and Future Work	23
	List of Figures	24
	List of Tables	25
	List of Listings	26
	Bibliography	27

Introduction

1.1 Context and Motivation

Active mechatronic devices such as powered prostheses, exoskeletons, and robotic assistive systems combine mechanical structures, electrical actuators, sensors, embedded controllers, and software into tightly coupled multi-domain systems. They are typical examples of cyber-physical systems, since computational functions and physical processes interact continuously through sensing, actuation, and feedback [1]. Changes in one subsystem, for example in the control law, actuator design, or joint geometry, can affect loads, stability, performance, and safety of the overall device. Understanding and validating such interactions therefore requires system-level simulation already in early design phases, before physical prototypes are available [2].

For devices that interface with the human body, this challenge becomes even more demanding. A prosthetic knee or an active exoskeleton is not merely a technical device. Its function depends on the coupled dynamics of the device and the musculoskeletal system of the user. In addition, local structural effects such as compliance, deformation, and contact may become relevant in critical operating phases. Questions such as whether a controller remains stable during ground contact, how loads are transferred through compliant components, or how device forces interact with biological tissue cannot be answered from isolated component analyses alone. They require virtual testing environments that represent both the technical system and its biomechanical context in a single, time-synchronous simulation [3].

In engineering practice, however, the required subsystem models are rarely created within a single tool. Mechanical and structural behavior is often derived from geometry-based models in computer-aided design and finite element analysis environments. Control-oriented, electrical, and other system-level subsystems are commonly formulated as one-dimensional equation-based models, for example in Modelica [4]. Musculoskeletal dynamics, in turn, are modeled in specialized biomechanics software such as OpenSim [5]. These modeling environments serve different purposes, operate at different abstraction levels, and use different numerical representations. Modern system development therefore requires not only reusable models within individual domains, but also a practical way to integrate heterogeneous models into a coherent system-level simulation workflow.

Co-simulation provides a natural integration strategy for such heterogeneous models. Each subsystem retains its domain-specific solver and model representation, while a master algorithm orchestrates data exchange at discrete communication points [6]. The Functional Mock-up Interface (FMI) standardizes the component interface for this purpose and is widely adopted in industry [7].

However, FMI defines the *interface* to individual simulation components, but it does not prescribe how multiple components are orchestrated into a consistent system simulation [6]. Several challenges remain when heterogeneous models are coupled in practice. These challenges include determining a correct execution order when component outputs depend algebraically on inputs [8], detecting and resolving circular dependencies between components [9], and handling discrete events such as mechanical contact or discrete mode changes during otherwise continuous operation [10–12].

Moreover, different representations of the same physical subsystem involve different trade-offs between computational cost and the level of detail provided by the simulation. A rigid-body model computes global kinematics and kinetics efficiently, but it does not explicitly resolve local deformation or stress. A finite element model can resolve these local field quantities, but at significantly higher computational cost. In system simulation, this suggests that detailed models should be activated selectively. For example, a finite element representation may only be needed during contact phases in which local deformation is relevant, whereas a computationally inexpensive rigid-body model may be sufficient during free motion. To enable such selective model use, the simulation framework must support switching between alternative models of the same subsystem at runtime, together with consistent state transfer across model boundaries [13].

1.2 State of the Art

This section positions the thesis within the current landscape of modeling and simulation approaches relevant to heterogeneous hybrid system simulation. It does not aim to provide an exhaustive review. Instead, it highlights the main modeling paradigms, tool classes, and orchestration challenges that form the context for the development of *SysSimX*.

1.2.1 Equation-based Modeling and FMI-based Components

Equation-based modeling describes systems through mathematical relations between physical quantities. In simulation, these relations are formulated as algebraic equations, differential equations, or combinations of both. Two main modeling styles can be distinguished. In causal modeling, the computational direction is defined explicitly through inputs and outputs. In acausal modeling, the relations between variables are stated as equations, and the simulation tool derives the required computational causality. Both approaches are relevant for cyber-physical and mechatronic systems. Causal models are common for control, signal processing, and software-oriented subsystems. Acausal equation-based models are especially useful for physical subsystems whose behavior follows conservation laws and constitutive

relations.

Equation-based modeling has become an established approach for the description of multi-domain physical systems. It allows engineers to formulate subsystem behavior in a compact and domain-oriented way without prescribing the full numerical solution procedure. This is particularly useful for mechatronic systems, since such systems combine mechanics, electronics, thermal effects, hydraulics, and control-oriented behavior. A common formalism helps to represent these domains within one model structure and to connect them through physically meaningful interfaces.

The most prominent open representative of this paradigm is Modelica [4]. Modelica is an object-oriented, equation-based language for the modeling of physical systems. It supports hierarchical composition, reusable component libraries, and acausal connectors. Through these features, it is widely used for the modeling of multi-domain systems in both academia and industry. Commercial and open-source environments such as Dymola, SimulationX, and OpenModelica provide solvers, graphical editors, and libraries based on the Modelica language and the Modelica Standard Library.

For tool interoperability, the Functional Mock-up Interface (FMI) standard has become particularly important [7]. It provides a standardized way to package and exchange simulation models across tools and therefore plays a central role in component-based simulation workflows [6].

Equation-based tools and FMI-based components provide a strong basis for system simulation. However, they do not cover all domain-specific modeling needs equally well.

1.2.2 Domain-Specific Simulation Environments

Although equation-based modeling environments cover many physical domains, they do not represent all simulation needs equally well. In particular, some subsystem classes require specialized modeling approaches and dedicated software tools. This is especially relevant for the systems considered in this thesis, where technical subsystems interact with the human musculoskeletal system and where local structural effects may become important.

A first example is musculoskeletal simulation. In applications such as prostheses and exoskeletons, the technical device is mechanically coupled to the human body [3]. The resulting system behavior depends not only on the device itself, but also on the biomechanics of the user. For such questions, musculoskeletal simulation environments provide modeling capabilities that go beyond classical one-dimensional component libraries. OpenSim is a widely used open platform in this area [5, 14]. It supports the modeling of articulated rigid-body systems with joints, muscle-tendon actuation, and biomechanical constraints. It also provides established workflows such as forward dynamics and inverse dynamics. In this way, OpenSim enables analyses that are highly relevant for human-device interaction, but it is not primarily designed as a standardized FMI-based component source.

A second example is finite element analysis. Finite element models are used when spatially distributed effects must be resolved explicitly. This includes deformation, stress, strain, and contact phenomena. Such information cannot usually be obtained from lumped-

parameter rigid-body models alone. Finite element methods solve partial differential equations on a spatial discretization of the geometry and therefore provide a much higher level of local detail [15]. This is important when compliant components, structural loading, or local contact behavior must be analyzed.

Musculoskeletal models and finite element models differ substantially from classical equation-based system components. Their interfaces are often not defined through a small set of reusable connectors. Instead, coupling variables must be chosen specifically for the application. These variables may include generalized coordinates, torques, external forces, contact quantities, marker trajectories, or reduced state representations. As a result, such models are usually integrated through custom wrappers or tool-specific interfaces rather than through direct FMI export.

For heterogeneous system simulation, this creates an important challenge. Domain-specific tools provide essential high-fidelity models, but they are not naturally embedded in the standardized component-oriented workflows that are common in FMI-based environments. A practical system simulation framework must therefore be able to integrate both standardized FMU-based models and non-FMI domain-specific models within one common orchestration concept.

1.2.3 Co-Simulation Frameworks and Orchestration

Co-simulation enables the coupled simulation of subsystem models that originate from different tools. In this setting, each subsystem keeps its own model representation and, in many cases, its own numerical solver. The coupled simulation is coordinated through data exchange at communication points. This makes co-simulation a natural approach for heterogeneous system simulation.

The most important standard in this area is the Functional Mock-up Interface (FMI) [7, 16]. FMI defines a standardized interface for simulation components called Functional Mock-up Units (FMUs). An importing environment creates FMU instances and calls their standardized interface functions. In the case of Model Exchange, the importing tool provides the numerical solver and performs the integration. In the case of Co-Simulation, the FMU typically advances its own internal solver and exchanges values only at communication points. FMI 3 further extends this concept through Scheduled Execution and explicit clock-based mechanisms for discrete-time behavior and event-oriented interaction [16].

Building on FMI, several frameworks have been developed to orchestrate coupled FMU execution. Standalone tools such as OMSimulator provide FMI-based composition and execution environments with scripting and graphical support [17]. INTO-CPS Maestro provides another orchestration environment for FMI-based co-simulation and supports the specification and execution of coupled simulation scenarios [18]. These tools show that FMI-based co-simulation is well established at the level of standardized component coupling.

Within the Python ecosystem, several libraries address FMI at different abstraction levels. FMPy provides lightweight support for loading, inspecting, validating, and simulating FMUs [19]. PyFMI offers a more simulation-oriented Python interface for Model Exchange and

Co-Simulation FMUs [20]. Research-oriented tools such as CoFmuPy further show that lightweight Python-based co-simulation environments can be built around FMI-compliant components [21]. Taken together, these tools show that FMI-based orchestration is mature for FMU-centric workflows. This is especially true when subsystem models are available through standardized component interfaces.

At the same time, the scope of most existing co-simulation frameworks remains strongly FMU-centered. Their direct interoperability is strongest when subsystem models are available as FMI-compliant components. Models from tools that do not naturally export FMUs, such as musculoskeletal simulation environments or finite element solvers, usually require custom wrappers or tool-specific interfaces. In addition, FMI standardizes the component interface, but it does not by itself solve all orchestration problems at system level. Questions such as execution order, algebraic-loop handling, hybrid event handling, and runtime switching between alternative subsystem models remain the responsibility of the importing framework. For this reason, current co-simulation frameworks provide an essential basis for interoperability. However, they do not fully address the broader requirements of heterogeneous hybrid system simulation.

1.2.4 Hybrid Co-Simulation and Event Handling

Hybrid systems combine continuous-time dynamics with discrete changes of state or behavior. In cyber-physical and mechatronic systems, such behavior arises in several ways. Typical examples are sampled sensors, digital controllers, switching logic, mode changes, and mechanical contact. Their simulation therefore requires both the numerical solution of the continuous-time problem and the consistent handling of discrete events.

In a monolithic simulation, one solver has access to the full system model. It can evaluate all relevant variables and equations during each integration step. This makes it possible to detect event conditions, localize the event time, execute the required event logic, and then continue the continuous integration from a consistent post-event state [4]. For hybrid systems, this is a major advantage of a monolithic formulation.

In co-simulation, the situation is more difficult. The master algorithm usually sees subsystem outputs only at communication points, while each component advances internally between these points. As a result, an event may occur within a communication step without being immediately visible at system level. Hybrid co-simulation therefore requires additional orchestration mechanisms. The importing framework must be able to detect that an event has occurred, localize the event time, restore or roll back subsystem states when necessary, execute the event handling logic, synchronize the affected components, and only then resume the continuous simulation [10]. The problem becomes even more difficult when several events interact or when one event triggers further events in other subsystems.

This challenge has been discussed extensively in the co-simulation literature. In particular, the representation of time, event handling, and rollback capabilities have been identified as central issues for deterministic and robust hybrid co-simulation [8, 11, 12]. FMI 3 extends FMI 2 with additional concepts that are relevant in this context, including clocks,

Event Mode, and Scheduled Execution [16]. These additions improve the representation of sampled and event-driven behavior. However, the standard alone does not solve the full orchestration problem. The importing framework must still implement the system-level logic for event detection, event localization, state consistency, and continuation after events.

Hybrid co-simulation is therefore a central topic for the present thesis. The systems considered here include both sampled and event-driven behavior, for example sensor sampling, controller action, and mechanical contact. A heterogeneous simulation framework must be able to combine continuous integration with robust event handling across coupled subsystems.

1.2.5 Multi-Model and Multi-Fidelity Simulation

Many engineering subsystems can be represented by more than one simulation model. These models may differ in their physical assumptions, numerical complexity, computational cost, and output detail. Model fidelity denotes how closely a model represents the system properties of interest [22]. This leads to the idea of multi-model or multi-fidelity simulation [23, 24].

A low-dimensional or reduced-complexity model is often sufficient when the objective is to predict global system behavior, design controllers, or perform rapid parameter studies. More detailed models are needed when local effects become important, for example deformation, stress, or contact. The appropriate model therefore depends on the simulation objective and on the current operating phase of the system [13, 25].

In engineering practice, this trade-off appears both across the development process and during runtime. Early design stages often rely on simple models because they are fast to build and simulate. Later stages require more detailed models for validation. A similar principle can also be used within one simulation scenario, where different phases of motion require different levels of detail.

Region-based runtime model substitution has been proposed as a strategy for simulation speed-up. Choi et al. describe a multi-fidelity methodology in which a computationally expensive target model is replaced by lower-fidelity models outside an interest region while preserving a common external interface and switching between internal models according to fidelity-change conditions [13, 26]. The present thesis is conceptually related to this idea, but applies it in a narrower sense. The framework switches between heterogeneous subsystem models from different tools that share a common external interface, rather than between simplified derivatives of one target model.

Existing work on multi-fidelity simulation covers a broad range of approaches, including hierarchical model families, reduced-order models, surrogate models, and adaptive model selection [23, 24]. For the present thesis, the most relevant aspect is the practical switching between alternative subsystem models within one simulation workflow. In this setting, the main challenge is not only the existence of models with different fidelity, but also the consistent transition between them. The framework must preserve a common external interface, transfer the relevant state information, define suitable switching criteria, and avoid unstable

or excessively frequent mode changes [25]. In heterogeneous settings, such switching typically requires projection onto a shared reduced state rather than exact preservation of all internal model variables.

1.3 Problem Statement and Research Gap

Equation-based simulation, musculoskeletal simulation, finite element analysis, and FMI-based co-simulation are well established within their respective domains. However, many mechatronic and biomechanical systems require these modeling paradigms to be combined within one system-level simulation workflow. In practice, such integration remains difficult. Existing approaches are strong either in standardized FMU-based coupling or in domain-specific analysis, but they provide only limited support for the unified integration of FMU models, musculoskeletal models, and finite-element models within one common simulation environment.

This limitation becomes particularly relevant when heterogeneous tool coupling must also be combined with dependency-aware execution, algebraic-loop handling, hybrid event handling, and runtime switching between alternative subsystem models. In this area, a practical and reproducible workflow is still insufficiently addressed. As a result, the analysis of systems whose relevant physics and appropriate model representation change across operating phases remains inconvenient, for example when transitioning between free motion, human interaction, and contact-dominated behavior.

This thesis addresses this methodological gap through the development and evaluation of the *SysSimX* framework for heterogeneous hybrid system simulation. More specifically, the thesis makes the following contributions:

- the definition of a unified component-based abstraction for heterogeneous subsystem models,
- the development of a graph-based orchestration approach with dependency analysis, execution-order resolution, and algebraic-loop handling,
- the integration of hybrid event handling within a common co-simulation workflow,
- the introduction of a multi-model component concept for runtime switching between alternative subsystem models with consistent state synchronization, and
- the implementation and evaluation of a controlled-pendulum case study combining FMU, OpenSim, and FEM models.

1.4 Research Questions

This thesis investigates how heterogeneous subsystem models from different simulation environments can be integrated into a coherent system-level simulation workflow. The central research question is formulated as follows:

How can a framework support heterogeneous hybrid co-simulation of subsystem models from different tools while enabling consistent orchestration, event handling, and runtime switching between alternative models of the same subsystem?

To answer this main question, the following sub-questions are addressed:

- RQ1** How can heterogeneous simulation models from equation-based, musculoskeletal, and finite element tools be represented through a unified component interface for co-simulation?
- RQ2** How can dependencies between heterogeneous components be analyzed in order to derive a consistent execution order and to identify direct feedthrough and algebraic loops?
- RQ3** How can hybrid phenomena, such as contact, switching, and discrete state changes, be detected, localized, and handled within the same co-simulation workflow?
- RQ4** How can runtime switching between alternative subsystem models, including low-dimensional and more detailed representations, be achieved with sufficiently consistent state transfer?
- RQ5** To what extent can the resulting framework reproduce relevant system behavior and support representative benchmark scenarios such as the controlled pendulum?

1.5 Objectives

Based on these research questions, the overall objective of this thesis is to develop and evaluate the *SysSimX* framework as a practical approach for heterogeneous hybrid system simulation. In particular, the thesis pursues the following objectives:

- O1** Design a unified component-based abstraction for heterogeneous subsystem models.
- O2** Develop a graph-based orchestration concept for coupled system models, including dependency analysis and execution-order resolution.
- O3** Implement support for direct feedthrough analysis and algebraic-loop handling within the orchestration workflow.
- O4** Integrate hybrid event handling, including event detection, event localization, and event execution, into the framework.
- O5** Develop a multi-model component concept that enables runtime switching between alternative subsystem models with consistent state synchronization.
- O6** Apply the framework to a controlled-pendulum case study combining FMU, OpenSim, and FEM models.
- O7** Evaluate the framework with respect to functional correctness, consistency of coupled behavior, and practical applicability in representative simulation scenarios.

1.6 Thesis Outline

Theoretical Background

Purpose: Only the theory that is prerequisite to understanding multiple features. Written to be read once, front-to-back.

2.1 Continuous-Time System Modeling

2.1.1 Dynamical Systems with Inputs and Outputs

A model is a mathematical representation of a real system. It is used to describe the system behavior and to answer questions by analysis or simulation. The same physical system can be represented by different models. The choice of model depends on the question of interest and on the required level of detail [27, 28].

A dynamical system is a system whose behavior changes over time. In such systems, the effect of an action does not occur instantaneously. Instead, the future behavior depends on the current condition of the system and on external influences. Examples are mechanical motion, electrical circuits, and thermal processes [27].

In engineering, complex systems are usually described as compositions of smaller subsystems or components. Each component interacts with its environment through inputs and outputs. Inputs represent external influences acting on the component. Outputs represent variables that are provided to other components or to the environment. This input-output view is useful because it supports the modular description of larger systems from connected subsystems [28].

In addition to inputs and outputs, a dynamical system is characterized by its state. The state is a set of variables that contains the information required to predict the future evolution of the system for a given input. The set of all possible states is called the state space. For physical systems, state variables often represent quantities such as position, velocity, momentum, or stored energy [27, 28].

A common representation of continuous-time dynamical systems is the state-space form

$$\dot{x} = f(x, u, t), \quad y = h(x, u, t), \quad (2.1)$$

where x is the state vector, u is the input vector, and y is the output vector. The function f describes the time evolution of the state. The function h defines how the outputs are

obtained from the state and the inputs. If the functions do not depend explicitly on time, the system is called time-invariant [27].

As a simple example, consider an RC circuit with input voltage $u(t)$ and capacitor voltage $v_C(t)$. The capacitor voltage stores the past effect of the input and is therefore a natural state variable. With

$$x(t) = v_C(t),$$

the system can be written as

$$\dot{x} = -\frac{1}{RC}x + \frac{1}{RC}u, \quad y = x. \quad (2.2)$$

This is a continuous-time system with one state, one input, and one output. Its future behavior depends on the current capacitor voltage and on the applied input voltage.

For linear time-invariant systems, the state-space form reduces to

$$\dot{x} = Ax + Bu, \quad y = Cx + Du, \quad (2.3)$$

where A , B , C , and D are constant matrices. This representation is widely used in systems and control theory. It also makes clear that the output may depend not only on the state, but also directly on the input through the matrix D . This property is important in co-simulation, because it is closely related to the concept of direct feedthrough discussed later in this thesis [27].

2.1.2 Ordinary and Differential-Algebraic Equations

Many continuous-time systems can be described by ordinary differential equations. In this case, the time derivative of the state is given explicitly as a function of the current state, the input, and time [29, 30]. A general nonlinear ODE can be written as

$$\dot{x} = f(x, u, t), \quad (2.4)$$

where x is the state vector and u is the input vector.

However, not all systems can be written naturally in explicit ODE form. In many physical and component-based models, additional algebraic constraints are present. These constraints may arise from constitutive relations, conservation laws, interface conditions, or the interconnection of subsystems [28, 30]. In such cases, the model is more naturally written as a differential-algebraic equation.

A general DAE can be written in implicit form as

$$F(t, z, \dot{z}, u) = 0, \quad (2.5)$$

where z contains the model variables and u denotes the input. In contrast to an ODE, the derivative is not given explicitly. Instead, differential and algebraic relations are stated together [29].

A common semi-explicit form of a DAE is

$$\dot{x} = f(x, w, u, t), \quad (2.6)$$

$$0 = g(x, w, u, t), \quad (2.7)$$

where x contains the differential states and w contains algebraic variables. The first equation describes the time evolution of the states. The second equation imposes algebraic constraints that must hold at every time instant. This form is useful because it separates dynamic behavior from purely algebraic relations [29, 30].

In contrast to ODEs, a DAE does not in general admit arbitrary initial values. The initial state must be consistent with the algebraic constraints. In addition, DAEs are often classified by an index that reflects the structural difficulty of the problem. Higher-index systems are generally harder to analyse and simulate [29].

DAEs are especially important in component-based and equation-based modelling. When subsystems are connected, additional interface equations are introduced. As a result, variables that appear to be independent in isolated subsystem models may become constrained in the coupled system. For this reason, DAE formulations are often more suitable than pure ODE formulations for multi-domain physical systems [28, 30].

This is particularly relevant for modern modeling and simulation workflows. Subsystem models are often created separately and then coupled through algebraic constraints. In such settings, it is usually preferable to treat the DAE directly rather than to transform it manually into an ODE [29].

This distinction is relevant for the present thesis. Equation-based models exported as FMUs may contain direct dependencies and algebraic relations at their interfaces. These relations influence execution ordering, direct feedthrough analysis, and the treatment of algebraic loops in co-simulation. For this reason, ODE and DAE models form the mathematical basis for the system representations considered in *SysSimX*.

2.1.3 Direct Feedthrough and Interface Variables

When a system is decomposed into subsystems, each subsystem interacts with its environment through interface variables. These variables describe the quantities that cross the subsystem boundary. In a general systems view, they are represented as inputs and outputs. In physical systems, they may correspond to quantities such as force and velocity, voltage and current, or more abstract control and measurement signals [27, 28].

The input-output view is useful because it allows complex systems to be described as networks of connected subsystems. Each subsystem can then be studied and implemented as a separate model, while the overall behavior is obtained from the interconnection of these models [27, 28]. This is a central idea in component-based modeling and simulation.

Consider a subsystem in state-space form

$$\dot{x} = f(x, u, t), \quad y = h(x, u, t), \quad (2.8)$$

where x is the state vector, u is the input vector, and y is the output vector. A subsystem has direct feedthrough if the current output depends directly on the current input. If the output depends only on the state, then no direct feedthrough is present. In the linear time-invariant case,

$$\dot{x} = Ax + Bu, \quad y = Cx + Du, \quad (2.9)$$

direct feedthrough is present if the matrix D is nonzero [27].

This distinction is important in interconnected systems. If the output of one subsystem depends instantaneously on its input, and this output is connected to the input of another subsystem, a zero-delay dependency is created. If such dependencies form a closed loop, the coupled system may no longer be representable as a pure ODE. Instead, algebraic equations must be solved together with the differential equations, which leads naturally to a DAE or an algebraic loop [27, 29, 30].

In component-based physical modeling, interface variables are therefore not only communication signals. They also define how subsystem equations are coupled across their boundaries. This is particularly relevant in equation-based and multi-domain models, where interface conditions may introduce algebraic constraints between connected subsystems [28, 30].

For the present thesis, direct feedthrough is an important structural property of simulation components. It determines whether current outputs can be evaluated independently of current inputs or whether instantaneous dependencies must be taken into account. This distinction is used later to derive execution order, identify zero-delay dependencies, and detect algebraic loops in coupled co-simulation systems.

2.2 Hybrid System Modeling

2.2.1 Hybrid System Representation

Cyber-physical systems combine continuous physical processes with digital control and software. Further, impacts, switching logic, and sampled controllers can cause abrupt changes in modes, derivatives, or states. These changes and transitions happen in a much faster way than the usual changes in the system and are therefore considered as instantaneous events. A purely continuous model cannot describe such behavior in a consistent way. Hybrid models therefore combine continuous dynamics with discrete change [31, 32].

In this thesis, we use the reduced hybrid representation

$$(q(t), x(t), z(t), u(t), y(t)), \quad (2.10)$$

where $q(t)$ is the discrete state, $x(t)$ is the continuous state, $z(t)$ collects algebraic variables, $u(t)$ is the input, and $y(t)$ is the output. The discrete state q may represent an operating mode, a Boolean logic state, or an internal controller state. This notation is a reduced representation inspired by formal hybrid modeling approaches such as Omola [33, 34].

Between events, the discrete state is constant and the continuous part follows the active

continuous-time model:

$$0 = F_{q(t)}(\dot{x}(t), x(t), z(t), u(t), t), \quad y(t) = h_{q(t)}(x(t), z(t), u(t), t), \quad t \neq t_e. \quad (2.11)$$

At an event time t_e , the model may change its discrete state and may reset selected continuous states:

$$x^+ = R_j(x^-, z^-, q^-, u^-, t_e), \quad q^+ = \delta_j(x^-, z^-, q^-, u^-, t_e). \quad (2.12)$$

The function $R_j(\cdot)$ computes the post-event values of the continuous states. The function $\delta_j(\cdot)$ updates the discrete state. Here, x^- and q^- denote the values immediately before the event, and x^+ and q^+ denote the values immediately after the event. The system trajectory is therefore piecewise continuous [35, 36].

Two event effects are important in this thesis. A switching event changes the active mode or equations, but the continuous state remains continuous. A reset event changes one or more continuous states instantaneously. The first changes the derivative, while the second produces a jump in the state trajectory [36].

A simple example is an integrator with reset:

$$\dot{x}(t) = 1, \quad y(t) = x(t). \quad (2.13)$$

When the state reaches a threshold, an event is triggered and the state is reset:

$$x \geq 1 \quad \Rightarrow \quad x^+ = 0. \quad (2.14)$$

The state evolves continuously until the threshold is reached. At the event time, the state jumps to zero. After the reset, continuous evolution starts again from the new state.

2.2.2 Events, Zero-Crossings, and State Re-Initialization

Hybrid models change their behavior at event times. Two event trigger types are common. Time events occur at known or scheduled instants. State events depend on the continuous trajectory and are therefore not known in advance [31, 36].

In this thesis, state events are represented by event indicators, also called zero-crossing functions,

$$g_j(x(t), z(t), q(t), u(t), t). \quad (2.15)$$

An event of type j is triggered when the indicator crosses zero. For example, the condition $x \geq x_{\max}$ can be written as

$$g(x) = x - x_{\max}. \quad (2.16)$$

The event occurs when $g(x) = 0$ [37].

In numerical simulation, the event time is usually not located exactly at the end of an integration step. The simulator first detects that a zero-crossing occurred within a time interval. It then localizes the event time inside this interval with a root-finding procedure. This avoids integrating across a discontinuity without resolving its time of occurrence [31,

33].

Once the event time t_e has been found, the update maps introduced in Section 2.2.1 are evaluated. The resulting post-event state must satisfy the active continuous and algebraic equations before simulation can continue. This step is called state re-initialization or restart [33, 38].

2.2.3 Time Semantics in Hybrid Simulation

Hybrid simulation alternates between phases of continuous evolution and discrete event handling. During continuous phases, the state evolves according to the active differential-algebraic equations. At event times, discrete updates may change the discrete state, reset continuous states, or trigger further events.

A purely real-valued notion of time is often not sufficient for this process. Several discrete updates may have to be executed at the same physical time. This occurs, for example, when one event triggers another event immediately, or when several event conditions become active at the same instant [38].

For this reason, hybrid simulation often uses *superdense time*. A time point is then represented by the pair

$$(t, n), \tag{2.17}$$

where $t \in \mathbb{R}_{\geq 0}$ denotes the physical time and $n \in \mathbb{N}_0$ denotes a logical micro-step index. The physical time identifies when the event occurs. The index n orders the sequence of discrete updates at that same physical time. An increase in n does not represent additional physical time. It only represents progress in the event iteration at fixed time t [10, 12].

Superdense time makes it possible to distinguish between the state before an event and the states after one or more discrete updates at the same physical time. Continuous integration resumes only after all event updates at the current physical time have been processed. This distinction is important for the present thesis, since the hybrid co-simulation algorithm introduced later first localizes an event in physical time and then performs ordered event handling at that same time instant.

2.3 Co-Simulation Principles

2.3.1 Functional Mock-up Interface and Functional Mock-up Units

2.3.2 Orchestrated Co-Simulation and Communication Steps

2.3.3 Dependency Analysis and Execution Order

2.3.4 Algebraic Loops in Coupled Systems

2.3.5 Hybrid Co-Simulation and Event Handling

2.4 Domain-Specific Modelling Foundations

2.4.1 Equation-Based Modelling with Modelica

Principles of Acausal, Equation-Based Modeling

Causal Component Interfaces

FMU Export

2.4.2 Musculoskeletal Modelling with OpenSim

Kinematic Chains

Forward Dynamics

Hunt-Crossles Contact

2.4.3 Finite Element Analysis with NGSolve

Deformable Body Dynamics

Newmark Integration

Torque Application

Contact Modeling using the Penalty Method

Requirements and Tool Selection

3.1 Requirements

Key principles:

- No mention of Modelica, OpenSim, or NGSolve (tool-agnostic), focus on tool capabilities
- *"The framework shall support equation-based modelling of multi-domain physical systems."*
- *"The framework shall support musculoskeletal biomechanical models with forward dynamic."*
- *"The framework shall support continuum-mechanical models with spatial discretization."*

3.1.1 Functional User Requirements

3.1.2 Non-Functional User Requirements

3.1.3 Derived System Requirements

Modelling

Simulation

Heterogeneity

Interaction

3.2 Tool Comparison

3.2.1 Equation-Based Modelling

- Modelica / OpenModelica

- Simulink /Simscape
- Brief comparison against requirements

3.2.2 Musculoskeletal Modelling

- Commercial: LifeMOD, AnyBody, SIMM
- Free: OpenSim
- Brief comparison against requirements

3.2.3 Finite Element Analysis Tools

- Calculix/PreProMax
- openCFS
- netgen/NGSolve
- Brief comparison against requirements

3.2.4 System Simulation and Co-Simulation Frameworks

- FMI Standard and FMI-based tools
- OMSimulator (Modelica-centric, FMI-based, no OpenSim/FEM)
- MAESTRO / INTO-CPS
- Brief comparison against requirements

Framework Architecture

Purpose: Short Chapter establishing the overall design.

4.1 Design Principles and Decisions

4.2 Layered Architecture

4.3 CoSimComponent Interface

4.3.1 Lifecycle

4.3.2 Ports

4.3.3 State Management

4.4 System Class

4.4.1 Component Registration

4.4.2 Connections

4.4.3 Graph Construction

4.5 Algorithm Selection

Feature Implementation and Verification

Each section follows the pattern:

1. **Need:** 1-paragraph back reference to a requirement (not a re-motivation)
2. **Theory:** Feature-specific technique
3. **Verification Example:** Minimal focused test (not the full case study)

5.1 Port System and Unit-Aware Connections

5.2 Component Wrappers

5.2.1 FMI 2.0 Co-Simulation Wrapper

5.2.2 Musculoskeletal Model Wrapper

5.2.3 Finite Element Model Wrapper

5.3 Structural Analysis and Execution Ordering

5.4 Master Algorithms

5.5 Algebraic Loop Resolution

5.6 Hybrid Co-Simulation Algorithm

5.7 Multi-Model Component and Mode Switching

5.8 System Graph Visualization

CHAPTER 6

Case Study: Controlled Pendulum System

6.1 Choice and Justification of the Case Study

6.2 System Engineering of the Controlled Pendulum

6.2.1 System Architecture and Signal Flow

6.2.2 PID Controller

6.2.3 BLDC Drive Model

6.2.4 Sensors and Quantization

6.3 Pendulum Models

6.3.1 Modelica Pendulum

6.3.2 OpenSim Pendulum

6.3.3 FEM Pendulum

1. Mesh generation and geometry
2. Neo-Hookean Material Law
3. Variational Formulation
4. Newmark Time Integration Scheme
5. Torque Application
6. Contact Boundary
7. Rigid Body Mappings

6.3.4 Multi-Model Pendulum

6.4 System Assembly and Configuration

6.4.1 Component Wiring and Connection Graph

6.4.2 Algebraic Loop Analysis

6.4.3 Monolithic Modelica Reference Model

CHAPTER 7

Discussion and Conclusion

7.1 Discussion of Results

7.2 Limitations

7.3 Conclusions

7.4 Outlook and Future Work

List of Figures

List of Tables

List of Listings

Bibliography

- [1] E. Lee, *Cyber Physical Systems: Design Challenges*, Journal Abbreviation: Electrical Engineering and Computer Sciences Pages: 369 Publication Title: Electrical Engineering and Computer Sciences, **2008**.
- [2] H. Van der Auweraer, J. Anthonis, S. De Bruyne, J. Leuridan, “Virtual engineering at work: The challenges for designing mechatronic products”, *ResearchGate*, DOI 10.1007/s00366-012-0286-6.
- [3] R. Eberle, M. Ebenbichler, B. Reimeir, L. Ralfs, R. Weidner, “Biomechanical Simulation Pipeline for Exoskeletons - A digital tool for the targeted development of support systems”, *Industry 4.0 Science* **2025**, 2025, DOI 10.30844/I4SE.25.5.30.
- [4] P. Fritzson, *Principles of object oriented modeling and simulation with Modelica 3.3: a cyber-physical approach*, 2nd edition, John Wiley & Sons Inc, Hoboken, New Jersey, **2015**.
- [5] S. L. Delp, F. C. Anderson, A. S. Arnold, P. Loan, A. Habib, C. T. John, E. Guendelman, D. G. Thelen, “OpenSim: Open-Source Software to Create and Analyze Dynamic Simulations of Movement”, *IEEE Transactions on Biomedical Engineering* **2007**, 54, 1940–1950.
- [6] C. Gomes, C. Thule, D. Broman, P. G. Larsen, H. Vangheluwe, “Co-Simulation: State of the Art”, *ACM Computing Surveys* **2019**, 51, 1–33.
- [7] Functional Mock-up Interface Specification 2.0.
- [8] D. Broman, L. Greenberg, E. A. Lee, M. Masin, S. Tripakis, M. Wetter in Proceedings of the 18th International Conference on Hybrid Systems: Computation and Control, ACM, Seattle Washington, **2015**, pp. 179–188.
- [9] S. Sicklinger, V. Belsky, B. Engelmann, H. Elmqvist, H. Olsson, R. Wüchner, K.-U. Bletzinger, “Interface Jacobian-based Co-Simulation”, *International Journal for Numerical Methods in Engineering* **2014**, 98, 418–444.
- [10] D. Broman, C. Brooks, L. Greenberg, E. A. Lee, M. Masin, S. Tripakis, M. Wetter in 2013 Proceedings of the International Conference on Embedded Software (EMSOFT), IEEE, Montreal, QC, Canada, **2013**, pp. 1–12.

- [11] F. Cremona, M. Lohstroh, D. Broman, M. Di Natale, E. A. Lee, S. Tripakis in 2016 ACM/IEEE International Conference on Formal Methods and Models for System Design (MEMOCODE), **2016**, pp. 173–183.
- [12] F. Cremona, M. Lohstroh, D. Broman, E. A. Lee, M. Masin, S. Tripakis, “Hybrid co-simulation: it’s about time”, *Software & Systems Modeling* **2019**, *18*, 1655–1679.
- [13] S. H. Choi, S. J. Lee, T. G. Kim, Multi-fidelity modeling & simulation methodology for simulation speed up | Proceedings of the 2nd ACM SIGSIM Conference on Principles of Advanced Discrete Simulation, en, **2014**.
- [14] A. Seth, J. L. Hicks, T. K. Uchida, A. Habib, C. L. Dembia, J. J. Dunne, C. F. Ong, M. S. DeMers, A. Rajagopal, M. Millard, S. R. Hamner, E. M. Arnold, J. R. Yong, S. K. Lakshmikanth, M. A. Sherman, J. P. Ku, S. L. Delp, “OpenSim: Simulating musculoskeletal dynamics and neuromuscular control to study human and animal movement”, *PLOS Computational Biology* **2018**, *14*, e1006223.
- [15] K. Knothe, H. Wessels, *Finite Elemente*, Springer Berlin Heidelberg, Berlin, Heidelberg, **2017**.
- [16] Functional Mock-up Interface Specification 3.0.2.
- [17] OMSimulator - Integrated FMI and TLM-based Co-simulation with Composite Model Editing and SSP, en.
- [18] C. Thule, K. Lausdahl, C. Gomes, G. Meisl, P. G. Larsen, “Maestro: The INTO-CPS co-simulation framework”, *Simulation Modelling Practice and Theory* **2019**, *92*, 45–61.
- [19] Home - FMPy.
- [20] C. Andersson, J. Åkesson, C. Führer, “PyFMI: A Python Package for Simulation of Coupled Dynamic Models with the Functional Mock-up Interface”.
- [21] C. Friedrich, A. Lombana, J. Fasquel, C. Schlick, N. Bennani, M. Mendil, “CoFmuPy: A Python Framework for Rapid Prototyping of FMI-based Digital Twins”.
- [22] *IEEE recommended practice for validation of computational electromagnetics computer modeling and simulations*, (Eds.: Institute of Electrical and Electronics Engineers, IEEE-SA Standards Board), Institute of Electrical and Electronics Engineers, New York, **2011**.
- [23] B. Peherstorfer, K. Willcox, M. Gunzburger, “Survey of Multifidelity Methods in Uncertainty Propagation, Inference, and Optimization”, *SIAM Review* **2018**, *60*, 550–591.
- [24] M. G. Fernández-Godino, “Review of multi-fidelity models”, *Advances in Computational Science and Engineering* **2023**, *1*, 351–400.
- [25] M. A. Williams, A. G. Alleyne in 2014 American Control Conference, ISSN: 2378-5861, **2014**, pp. 3104–3109.

- [26] S. H. Choi, K.-M. Seo, T. G. Kim, “Accelerated Simulation of Discrete Event Dynamic Systems via a Multi-Fidelity Modeling Framework”, *Applied Sciences* **2017**, 7, DOI 10.3390/app7101056.
- [27] K. J. Åström, R. M. Murray, *Feedback systems: an introduction for scientists and engineers*, Princeton University Press, Princeton, **2008**.
- [28] D. Karnopp, D. L. Margolis, R. C. Rosenberg, *System dynamics: modeling, simulation, and control of mechatronic systems*, 5. ed, Wiley, Hoboken, NJ, **2012**.
- [29] P. Kunkel, V. Mehrmann, *Differential-Algebraic Equations: Analysis and Numerical Solution*, 2nd ed., EMS Press, **2024**.
- [30] F. E. Cellier, E. Kofman, *Continuous system simulation*, Springer, New York, **2006**.
- [31] F. Cellier, “Combined continuous/discrete simulation: applications, techniques and tools”, **1986**, 24–33.
- [32] P. Derler, E. A. Lee, A. Sangiovanni Vincentelli, “Modeling Cyber–Physical Systems”, *Proceedings of the IEEE* **2012**, 100, 13–28.
- [33] M. Andersson in **1990**.
- [34] M. Andersson, Doctoral Thesis (monograph), Department of Automatic Control, **1994**.
- [35] A. Van Der Schaft, H. Schumacher, *An introduction to hybrid dynamical systems*, Springer London, London, **2000**.
- [36] M. Branicky, *General Hybrid Dynamical Systems: Modeling, Analysis, and Control*. Pages: 200, **1995**.
- [37] M. Branicky, S. Mattsson in **2006**, pp. 31–56.
- [38] A. Urquía Moraleda, C. Martín Villalba, *Modeling and simulation in engineering using Modelica*, Universidad Nacional de Educación a Distancia (España). Editorial, **2018**.